

```
import threading

class GenderNeutralBathroom:
    def __init__(self):
        self.lock = threading.Lock()
        self.condition = threading.Condition(self.lock)
        self.current_state = "Empty" # 状态: Empty/Women/Men
        self.women_count = 0
        self.men_count = 0

    def woman_wants_to_enter(self):
        with self.lock:
            while self.current_state == "Men":
                self.condition.wait()
            self.women_count += 1
            if self.current_state == "Empty":
                self.current_state = "Women"
            self.condition.notify_all()

    def man_wants_to_enter(self):
        with self.lock:
            while self.current_state == "Women":
                self.condition.wait()
            self.men_count += 1
            if self.current_state == "Empty":
                self.current_state = "Men"
            self.condition.notify_all()

    def woman_leaves(self):
        with self.lock:
            self.women_count -= 1
            if self.women_count == 0:
                self.current_state = "Empty"
                self.condition.notify_all()

    def man_leaves(self):
        with self.lock:
            self.men_count -= 1
            if self.men_count == 0:
                self.current_state = "Empty"
                self.condition.notify_all()
```

测试示例

```
bathroom = GenderNeutralBathroom()
```

```
def woman_behavior():
```

```
    bathroom.woman_wants_to_enter()
```

```
    print("Woman entered")
```

```
    # 模拟使用洗手间
```

```
    bathroom.woman_leaves()
```

```
    print("Woman left")
```

```
def man_behavior():
```

```
    bathroom.man_wants_to_enter()
```

```
    print("Man entered")
```

```
    # 模拟使用洗手间
```

```
    bathroom.man_leaves()
```

```
    print("Man left")
```

模拟并发访问

```
threads = []
```

```
for _ in range(3):
```

```
    t = threading.Thread(target=woman_behavior)
```

```
    threads.append(t)
```

```
for _ in range(2):
```

```
    t = threading.Thread(target=man_behavior)
```

```
    threads.append(t)
```

```
for t in threads:
```

```
    t.start()
```

```
for t in threads:
```

```
    t.join()
```